

1.

Online C Compiler

Share

Settings

Save

Run

```

1 #include <stdio.h>
2
3 void printBinary(int n)
4 {
5     for (int i = 7; i >= 0; i--)
6         printf("%d", (n >> i) & 1);
7 }
8
9 int main()
10 {
11     int a, b;
12
13     printf("Enter two signed 8-bit numbers: ");
14     scanf("%d %d", &a, &b);
15
16     if (a < -128 || a > 127 || b < -128 || b > 127)
17     {
18         printf("Invalid input! Range is -128 to 127.\n");
19         return 0;
20     }
21
22     int result = (a + b) & 0xFF;
23
24     int signedResult = result;
25
26     if (result & 0x80)
27         signedResult = result - 256;
28
29     printf("\nBinary of %d = ", a);
30     printBinary(a & 0xFF);

```

Output

Status: **Accepted**

Memory Used: 720 byte | Execution Time: 0.003 sec

Enter two signed 8-bit numbers:
Binary of 50 = 00110010
Binary of 120 = 01111000

Binary result = 10101010
Signed result = -86
OVERFLOW DETECTED!
Actual mathematical result = 170

Input

50 120

2.

Online C Compiler

Share

Settings

Save

Run

```

1 #include <stdio.h>
2
3 void printBinary(int x, int bits)
4 {
5     for (int i = bits - 1; i >= 0; i--)
6         printf("%d", (x >> i) & 1);
7 }
8
9 int main()
10 {
11     int M, Q;
12     int A = 0;
13     int Qminus1 = 0;
14
15     int n = 8;
16
17     printf("Enter multiplicand (-128 to 127): ");
18     scanf("%d", &M);
19
20     printf("Enter multiplier (-128 to 127): ");
21     scanf("%d", &Q);
22
23     if (M < -128 || M > 127 || Q < -128 || Q > 127)
24     {
25         printf("Invalid input! Enter values from -128 to 127.\n");
26         return 0;
27     }
28

```

Output

```

.....
Q = 11010111
Q-1 = 1

Step 8
Q0 = 1, Q-1 = 1
Operation: No operation
After arithmetic right shift:
A = 11111111
Q = 11101011
Q-1 = 1

=====
Final Product = -21
=====

```

Input

7 -3

3.

Online C Compiler

Share

Settings

File

Run

```

1 #include <stdio.h>
2 #include <float.h>
3
4 int main()
5 {
6     float largeF, smallF;
7     double largeD, smallD;
8
9     printf("Enter a large number: ");
10    scanf("%f", &largeF);
11
12    printf("Enter a small number: ");
13    scanf("%f", &smallF);
14
15    largeD = largeF;
16    smallD = smallF;
17
18    float floatResult = largeF + smallF;
19    double doubleResult = largeD + smallD;
20
21    printf("\n--- Single Precision ---\n");
22    printf("Large number = %.10f\n", largeF);
23    printf("Small number = %.10f\n", smallF);
24    printf("Result      = %.10f\n", floatResult);
25
26    printf("\n--- Double Precision ---\n");
27    printf("Large number = %.15f\n", largeD);
28    printf("Small number = %.15f\n", smallD);
29    printf("Result      = %.15f\n", doubleResult);
30

```

Output

Enter a large number: Enter a small number:

--- Single Precision ---

Large number = 1.0000000000

Small number = 1000000.0000000000

Result = 1000001.0000000000

--- Double Precision ---

Large number = 1.000000000000000

Small number = 1000000.000000000000000

Result = 1000001.000000000000000

Single precision: approximately 7 decimal digits

Double precision: approximately 15-16 decimal digits

Input

110000000

4.

Online C Compiler

Share

Settings

File

Run

```

1 #include <stdio.h>
2
3 void restoringDivision(int dividend, int divisor)
4 {
5     int quotient = 0;
6     int remainder = 0;
7
8     for (int i = 7; i >= 0; i--)
9     {
10         remainder = (remainder << 1) |
11                     ((dividend >> i) & 1);
12
13         if (remainder >= divisor)
14         {
15             remainder = remainder - divisor;
16             quotient = quotient | (1 << i);
17         }
18     }
19
20    printf("\n--- Restoring Division ---\n");
21    printf("Quotient = %d\n", quotient);
22    printf("Remainder = %d\n", remainder);
23
24
25    void nonRestoringDivision(int dividend, int divisor)
26    {
27        int quotient = 0;
28        int remainder = 0;
29
30        for (int i = 7; i >= 0; i--)
31        {
32            remainder = (remainder << 1) |
33                        ((dividend >> i) & 1);
34
35            if (remainder >= divisor)
36            {
37                remainder = remainder - divisor;
38                quotient = quotient | (1 << i);
39            }
40        }
41
42        printf("\n--- Non-Restoring Concept ---\n");
43        printf("Quotient = %d\n", quotient);
44        printf("Remainder = %d\n", remainder);
45
46
47        Optimization:
48        Non-restoring division avoids unnecessary restoration.
49

```

Memory Used: 720 byte

Execution Time: 0.003 sec

Enter dividend: Enter divisor:

--- Restoring Division ---

Quotient = 14

Remainder = 2

--- Non-Restoring Concept ---

Quotient = 14

Remainder = 2

Optimization:

Non-restoring division avoids unnecessary restoration.

Input

100 7

5.

Online C Compiler

Share

Run

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int A, B;
6     int P[8], G[8], C[9];
7     int sum[8];
8
9     printf("Enter two 8-bit numbers (0-255): ");
10    scanf("%d %d", &A, &B);
11
12    if (A < 0 || A > 255 || B < 0 || B > 255)
13    {
14        printf("Invalid input!\n");
15        return 0;
16    }
17
18    C[0] = 0;
19
20    for (int i = 0; i < 8; i++)
21    {
22        int a = (A >> i) & 1;
23        int b = (B >> i) & 1;
24
25        P[i] = a ^ b;
26        G[i] = a & b;
27
28        C[i + 1] = G[i] | (P[i] & C[i]);
29        sum[i] = P[i] ^ C[i];
30    }

```

Output

```

C2 = 0
C3 = 1
C4 = 0
C5 = 0
C6 = 0
C7 = 0
C8 = 0

Result = 250

--- Optimization ---
Ripple Carry Adder: Carry propagates from one stage to the next.
Carry Look-Ahead Adder: Carry is calculated using Generate and Propagate.
Therefore, CLA reduces carry propagation delay.

```

Input

100 150